f

UNIDADE 05

Deseño de Interfaces con Tailwind CSS

Obxectivos da unidade

- **Coñecer** e empregar Tailwind CSS para crear interfaces web modernas, responsive e coherentes, aplicando o enfoque ***utility-first***.
- **Integrar** Tailwind CSS nunha aplicación desenvolvida con Vue 3, empregando as súas clases utilitarias para modificar e adaptar a interface sen alterar a lóxica da aplicación.
- **Aplicar** boas prácticas de **deseño responsive** e reutilización de estilos na construción de compoñentes e interfaces web.

A.1. Introdución a Tailwind CSS

Na primeira unidade construímos unha aplicación funcional empregando Vue 3. Traballamos cos **compoñentes, a reactividade, o enrutamento, Axios, as operacións CRUD e a xestión do estado con Pinia**. A aplicación funciona correctamente, pero a súa interface visual aínda está construída empregando principalmente CSS tradicional.

Nesta unidade imos centrarnos precisamente nesa parte: o deseño da interface. Para iso imos empregar ***Tailwind CSS,*** un **framework CSS** baseado nun enfoque denominado ***utility-first,*** que nos permite construír interfaces directamente a partir de **pequenas clases que representan propiedades concretas de CSS.**

Por exemplo, empregando CSS tradicional poderiamos ter:

```
.boton {
  background-color: blue;
  color: white;
  padding: 0.5rem 1rem;
  border-radius: 0.5rem;
}
```

E no HTML:

```
<button class="boton">Gardar</button>
```

Con Tailwind podemos expresar esas mesmas propiedades directamente no elemento:

```
<button class="bg-blue-600 text-white px-4 py-2 rounded-lg">
  Gardar
</button>
```

Non estamos facendo nada que non puidésemos facer con CSS. A diferenza **está na forma de organizar e aplicar os estilos.**

Tailwind proporciona unha gran **colección de clases** utilitarias que podemos combinar para construír a interface:

bg-blue-600 → cor de fondo

text-white → cor do texto

p-4 → padding

mt-2 → marxe superior

rounded-lg → bordes redondeados

flex → display flex

gap-4 → separación entre elementos

Isto **permite construír unha interface de maneira rápida** e, sobre todo, sen ter que crear continuamente novas clases CSS para cada elemento.

É importante entender que Tailwind non substitúe os coñecementos de HTML e CSS. Pola contra, Tailwind utiliza os conceptos que xa coñecemos de CSS e proporciona unha **forma diferente de aplicalos.**

Por exemplo, se sabemos que:

display: flex;

justify-content: center;

align-items: center;

podemos utilizar en Tailwind:

* *

Por tanto, non necesitamos volver aprender Flexbox, Grid, marxes, padding, cores ou tipografía. O que imos aprender é como aplicar estes conceptos utilizando as clases utilitarias de Tailwind.

Isto é especialmente importante neste módulo, xa que os coñecementos básicos de HTML e CSS xa foron traballados no módulo de Linguaxes de Marcas e Sistemas de Xestión de Información de primeiro curso.

Unha das **principais vantaxes de Tailwind** é que permite desenvolver interfaces de **forma moi rápida,** mantendo os estilos próximos ao elemento ao que se aplican.

Ademais, facilita especialmente a **creación de interfaces responsive,** permitindo modificar o comportamento dos elementos segundo o tamaño da pantalla.

Por exemplo:

```
<div class="w-full md:w-1/2 lg:w-1/3">
```

Neste caso:

- *w-full → ocupa todo o ancho en pantallas pequenas.*
- *md:w-1/2 → a partir do breakpoint md, ocupa a metade.*
- *lg:w-1/3 → en pantallas grandes ocupa un terzo.*

Deste xeito podemos adaptar a interface a diferentes dispositivos sen escribir directamente media queries CSS.

Durante esta unidade non imos crear unha aplicación nova. Partiremos da aplicación desenvolvida na **UD1** e modificaremos progresivamente a súa interface empregando Tailwind.

IMPORTANTE. A presentación dunha aplicación e a súa lóxica son aspectos diferentes.

Na UD1 xa temos funcionando:



Nesta unidade non imos modificar esa lóxica. Imos actuar sobre a capa visual.

A.2. Instalación e configuración de Tailwind CSS

Instalaremos Tailwind sobre o proxecto da UD1 e comprobaremos o seu funcionamento antes de comezar a modificar os distintos compoñentes.

Tailwind CSS funciona xerando os estilos correspondentes ás clases que utilizamos nos ficheiros HTML e nos compoñentes da aplicación. Deste xeito, as clases que empregamos no template convértense no CSS necesario durante o proceso de compilación.

Paso 1. Instalación

Desde o **directorio raíz** do proxecto executamos:

```
PS C:\Users\jcarl\proxectovue> npm install tailwindcss @tailwindcss/vite

added 15 packages, and audited 263 packages in 5s

55 packages are looking for funding
  run `npm fund` for details
```

A instalación incorpora dous elementos:

- ***tailwindcss***: o propio framework.
- ***@tailwindcss/vite***: o plugin que **permite integrar Tailwind** directamente co sistema de compilación de Vite.

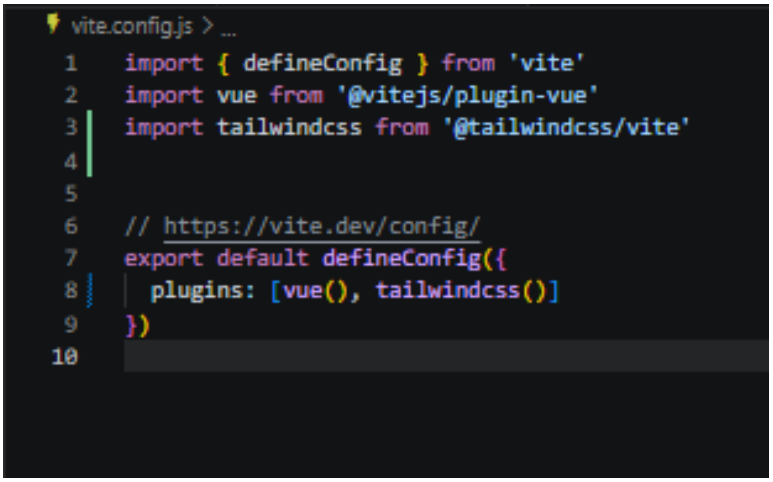
Esta é actualmente a forma recomendada para utilizar Tailwind con Vite.

Paso 2. Configuración de Vite

Agora temos que indicar a Vite que debe utilizar o plugin de Tailwind. No arquivo ***vite.config.js*** engadimos a importación:

```
import tailwindcss from '@tailwindcss/vite'
```

E incorporamos o plugin na configuración. O resultado será similar ao seguinte:



Non debemos eliminar o plugin de Vue. **Tailwind engádese á configuración** existente, xa que Vite necesita continuar utilizando **@vitejs/plugin-vue** para procesar os nosos compoñentes Vue.

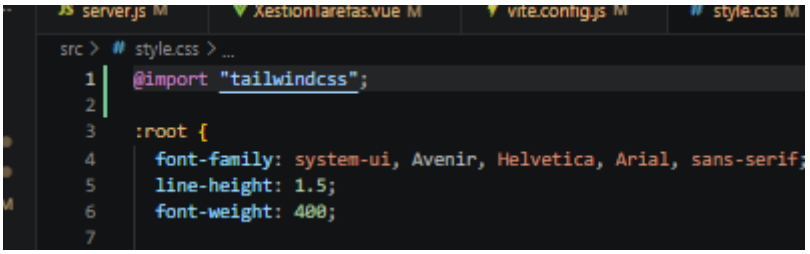
Paso 3. Incorporar Tailwind ao CSS

A continuación **imos ao arquivo CSS global** da aplicación, que no noso proxecto é:

src/style.css

Ao principio do arquivo engadimos:

@import "tailwindcss";



Esta única instrución incorpora Tailwind ao proxecto. Na versión actual xa non necesitamos engadir as **antigas directivas:**

~~*@tailwind base;*~~

~~*@tailwind components;*~~

~~*@tailwind utilities;*~~

Estas pertencen ao sistema utilizado en versións anteriores de Tailwind. **A versión 4 introduciu unha configuración moito máis sinxela baseada en CSS** e eliminou boa parte da configuración que era necesaria anteriormente.

Paso 4. Comprobar que Tailwind funciona

Antes de modificar a nosa aplicación imos facer unha pequena proba, por exemplo, *Sobrenos.vue*. No template dun dos nosos compoñentes podemos engadir temporalmente:

```
<h1 class="text-3xl font-bold underline">
    Tailwind funciona!
</h1>
```

Se iniciamos a aplicación:

```
npm start
```

deberemos observar o texto con maior tamaño, en negriña e subliñado. Isto confirma que Tailwind está correctamente instalado e integrado con Vite.



Importante. Nesta unidade veremos diferentes clases de Tailwind, p**ero non é necesario memorizar todas** as clases existentes.

Tailwind dispón dunha gra**n cantidade de utilidades e a** documentación permite consultar rapidamente a clase correspondente a cada necesidade.

O importante é **comprender a súa filosofía e aprender a identificar a relación entre unha propiedade CSS e a clase de Tailwind** que podemos empregar para aplicala.

Por exemplo:

CSS	Tailwind
display: flex	flex
padding: 1rem	p-4
margin-top: 0.5rem	mt-2
text-align: center	text-center
font-weight: bold	font-bold
border-radius	rounded

A partir deste momento xa podemos empezar a **substituír progresivamente o CSS** existente da nosa aplicación polas clases utilitarias de Tailwind.

Non imos aprender Tailwind para facer unha aplicación diferente. Imos aprender Tailwind para transformar a aplicación que xa construímos na UD1.

A.3. Clases utilitarias básicas

Unha vez instalado e configurado Tailwind CSS, xa podemos comezar a utilizalo nos compoñentes da nosa aplicación.

Como vimos no apartado anterior, Tailwind baséase no uso de **clases utilitarias**. **Cada clase representa,** normalmente, **unha propiedade ou un pequeno conxunto de propiedades CSS.**

Por exemplo:

```
<button class="bg-blue-600 text-white px-4 py-2 rounded">
  Gardar
</button>
```

Neste caso non estamos creando unha clase CSS chamada boton. Estamos **combinando varias clases de Tailwind:**

bg-blue-600 → *cor de fondo*

text-white → *cor do texto*

px-4 → *espazamento horizontal*

py-2 → *espazamento vertical*

rounded → *bordes redondeados*

Esta é a idea fundamental de Tailwind é **construír o aspecto dun elemento combinando pequenas clases que representan as súas características visuais.**

1. Cores

Tailwind proporciona clases para establecer cores de fondo, texto e bordes. Por exemplo:

```
<div class="bg-blue-600 text-white">
  Usuario activo
</div>
```

bg- fai referencia á cor de fondo (*background*) e **text**- á cor do texto.

As cores dispoñen normalmente de diferentes intensidades:

bg-blue-100

bg-blue-300

bg-blue-500

bg-blue-700

bg-blue-900

O **número indica a intensidade** aproximada da cor. Tamén podemos utilizar diferentes familias:

red green blue

yellow gray slate

indigo purple yellow ….

Por exemplo:

```
<span class="bg-green-100 text-green-800">
  Activo
</span>
```

ou:

```
<span class="bg-red-100 text-red-800">
  Inactivo
</span>
```

Isto encaixa directamente coa nosa aplicación, xa que na listaxe de usuarios xa mostramos visualmente se un usuario está activo ou non.

2. Tipografía

Tamén podemos modificar facilmente o aspecto do texto.

Por exemplo:

```
<h1 class="text-3xl font-bold">
  Xestión de usuarios
</h1>
```

Aquí:

text-3xl → *tamaño do texto*

font-bold → *texto en negriña*

Outras clases habituais son:

text-sm

text-base

text-lg

text-xl

text-2xl

text-3xl

E para o grosor:

font-light

font-normal

font-medium

font-semibold

font-bold

Tamén podemos aliñar o texto:

```
<p class="text-center">
  Lista de usuarios
</p>
```

As clases máis habituais son:

text-left text-center text-right

3. Espazado

Unha das partes que máis utilizaremos en Tailwind é o control do espazado. As clases **p-** fan referencia ao **padding** e as clases **m-** ao **margin**.

Por exemplo:

```
<div class="p-4">
   Contido
</div>
```

equivale aproximadamente a:

```
padding: 1rem;
```

Podemos indicar tamén unha dirección concreta:

pt-4 → *padding-top*

pb-4 → *padding-bottom*

pl-4 → *padding-left*

pr-4 → *padding-right*

Ou utilizar:

px-4 → *padding horizontal*

py-4 → *padding vertical*

Para as marxes empregamos o mesmo sistema:

m-4

mt-4

mb-4

ml-4

mr-4

mx-4

my-4

Por exemplo:

```
<div class="mt-4 mb-6">
    Formulario de usuarios
</div>
```

4. Tamaños

Tamén podemos establecer o ancho e o alto dos elementos. Por exemplo:

```
<div class="w-full">
```

indica que o elemento ocupa todo o ancho dispoñible. Outras posibilidades que utilizaremos máis adiante son:

w-full

w-1/2

w-1/3

w-1/4

Para a altura podemos utilizar:

h-full

h-10

h-12

h-16

Non é necesario memorizar todos estes valores. O importante **é comprender a convención de nomes** utilizada por Tailwind e saber consultar a documentación cando necesitemos unha medida concreta.

5. Bordes, esquinas e sombras

Tamén podemos modificar facilmente os bordes e o aspecto dos elementos.

Por exemplo:

```
<div class="border rounded-lg shadow p-4">
  Datos do usuario
</div>
```

Neste caso:

border → *engade un borde*

rounded-lg → *redondea as esquinas*

shadow → *engade unha sombra*

p-4 → *engade espazamento interior*

Podemos modificar o nivel de redondeo:

rounded

rounded-md

rounded-lg

rounded-xl

rounded-full

E tamén o tamaño da sombra:

shadow-sm

shadow

shadow-md

shadow-lg

6. Combinando clases

A **verdadeira potencia de Tailwind aparece cando combinamos** varias utilidades.

Por exemplo, podemos crear unha pequena tarxeta para mostrar un usuario:

```
<div class="bg-white p-6 rounded-lg shadow-md">
  <h2 class="text-xl font-bold">
    Juan Pérez
```

```
        </h2>

        <p class="text-gray-600 mt-2">
          juan@email.com
        </p>

      <span class="bg-green-100 text-green-800 px-3 py-1 rounded-full">
          Activo
        </span>
      </div>
```

Non creamos ningunha clase CSS propia. A aparencia da tarxeta está **construída combinando as diferentes utilidades.**

7. Tailwind aplicado ao noso proxecto

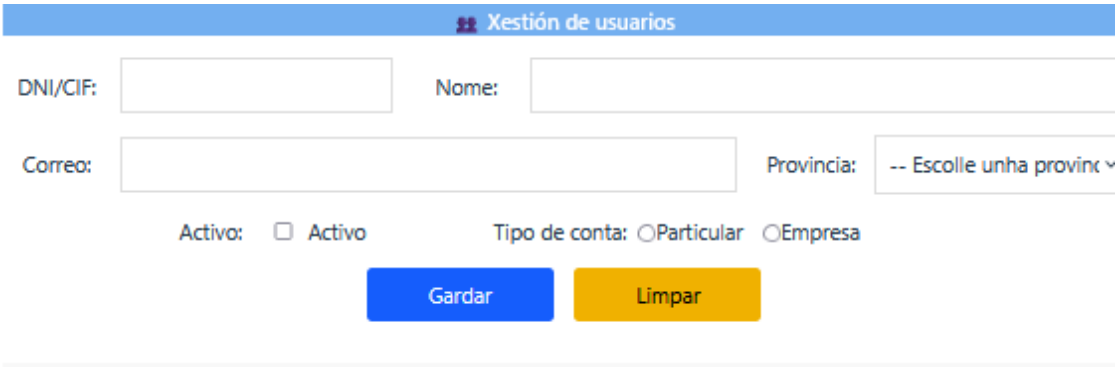
Agora que coñecemos as utilidades básicas, podemos comezar a modificar a aplicación da UD1. Por exemplo, na nosa aplicación temos botóns para **gardar, editar e eliminar usuarios**. Un botón que anteriormente utilizaba unha clase CSS propia podería pasar a ser:

```
<div class="flex justify-center gap-4">
    <button type="submit" class="w-1/6 bg-blue-600 text-white px-4 py-2 rounded" :disabled="novoUsuario.dni === '' || novoUsuario.nome === ''">
     Gardar </button>

   <button type="button" class="w-1/6 bg-yellow-500 text-black px-4 py-2 rounded" @click="limparForm">
     Limpar </button>
  </div>
```

Deste xeito empezamos a comprobar como podemos **substituír progresivamente o CSS existente da aplicación por clases utilitarias de Tailwind**.

Es importante eliminar clases de button o bien sus referencias para que se apliquen los estilos de Tailwind.



Non imos modificar aínda toda a aplicación. O obxectivo deste apartado é familiarizarnos co sistema de clases. Nos seguintes apartados iremos incorporando estas técnicas para construír o deseño completo.

Importante: Tailwind non elimina CSS. Simplemente proporciona unha **forma diferente de escribir e organizar os estilos,** utilizando clases utilitarias que podemos combinar directamente nos elementos da interface.

A.4. Deseño con Flexbox e Grid

En Tailwind CSS, a organización dos elementos realízase principalmente mediante as utilidades de ***Flexbox e CSS Grid.*** Estas ferramentas definen como se distribúen os elementos dun contedor sen necesidade de escribir regras CSS específicas.

Ata agora utilizamos clases para modificar propiedades individuais, como cores, espazado ou tamaño. Agora usaremos estas clases para **definir a estrutura e distribución dos elementos da interface**.

A.4.1. Flexbox

Flexbox resulta especialmente útil cando queremos organizar elementos nunha **fila ou nunha columna**. Para activar Flexbox simplemente utilizamos:

```
<div class="flex">
```

A partir dese momento podemos controlar a dirección, a distribución e a aliñación dos elementos interiores. As clases máis utilizadas son:

Clase	Función
flex	Activa Flexbox
flex-row	Distribúe os elementos en fila
flex-col	Distribúe os elementos en columna
justify-start	Aliña ao principio
justify-center	Centra horizontalmente
justify-end	Aliña ao final
justify-between	Distribúe deixando espazo entre elementos
items-start	Aliña verticalmente ao principio
items-center	Centra verticalmente
items-end	Aliña verticalmente ao final
*gap-**	Establece separación entre elementos

Por exemplo:

```
<div class="flex justify-center gap-4">
  <button class="bg-blue-600 text-white px-4 py-2 rounded">
    Gardar
  </button>
  <button class="bg-yellow-500 text-black px-4 py-2 rounded">
    Limpar
  </button>
</div>
```

Neste caso:

- *flex* activa Flexbox.
- *justify-center* centra os botóns.
- *gap-4* establece separación entre eles.

Un exemplo aplicado ao noso formulario

Na versión anterior tiñamos:

```
<div class="fila">
```

e no CSS:

```
.fila {
  display: flex;
  gap: 1rem;
  width: 100%;
}
```

Con Tailwind podemos eliminar esa regra CSS e escribir directamente:

```
<div class="flex gap-4 w-full">
```

Estamos facendo exactamente o mesmo, pero utilizando clases utilitarias. Isto permite que o estilo quede asociado directamente ao elemento que o utiliza e evita ter que crear una clase CSS específica como `.fila`.

A.4.2. Flexbox en columnas

Flexbox tamén permite **cambiar a dirección** dos elementos:

```
<div class="flex flex-col gap-4">
```

Neste caso:

- *flex* activa Flexbox.
- *flex-col* coloca os elementos verticalmente.
- *gap-4* establece separación entre eles.

É precisamente o que necesitamos, por exemplo, para organizar verticalmente determinados elementos dun formulario.

A.4.3. Distribución do espazo

Outra das posibilidades interesantes de *Flexbox* é controlar **como se reparte o espazo.**

Por exemplo:

```
<div class="flex justify-between">
  <span>Usuarios</span>
  <span>4 rexistros</span>
</div>
```

justify-between coloca o primeiro elemento ao principio e o segundo ao final do contedor.

Tamén podemos utilizar:

```
<div class="flex items-center">
```

para conseguir que os elementos queden **aliñados verticalmente** no centro.

Unha combinación moi habitual é:

```
<div class="flex items-center justify-between">
```

que permite crear, por exemplo, cabeceiras nas que temos un título á esquerda e unha acción á dereita.

A.4.4. CSS Grid

Flexbox é moi cómodo para organizar elementos principalmente nunha dimensión: **unha fila ou unha columna**.

Cando necesitamos organizar elementos simultaneamente en **filas e columnas**, CSS Grid resulta especialmente apropiado.

En Tailwind activamos Grid mediante:

```
<div class="grid">
```

E podemos indicar o número de columnas:

```
<div class="grid grid-cols-2 gap-4">
```

Neste caso teremos dúas columnas.

Por exemplo, podemos organizar un formulario así:

```
<div class="grid grid-cols-2 gap-4">

    <div>
      <label>DNI/CIF:</label>
      <input type="text">
    </div>

    <div>
      <label>Nome:</label>
      <input type="text">
    </div>

    <div>
      <label>Correo:</label>
      <input type="email">
    </div>

    <div>
```

```
            <label>Provincia:</label>
            <select>
              ...
            </select>
          </div>

        </div>
```

As clases principais serán:

Clase	Función
grid	Activa CSS Grid
grid-cols-1	Unha columna
grid-cols-2	Dúas columnas
grid-cols-3	Tres columnas
grid-cols-4	Catro columnas
*gap-**	Separación entre elementos

A.4.5. Flexbox ou Grid?

Non se trata de escoller unha ferramenta e abandonar a outra. **Ambas poden convivir na mesma interface.**

Como regra xeral:

- ***Flexbox*** resulta especialmente cómodo para **organizar elementos nunha fila ou columna.**
- ***Grid*** resulta especialmente útil para **estruturas baseadas en filas e columnas.**

Por exemplo, podemos utilizar Grid para organizar os campos dun formulario:

* *

e Flexbox dentro de cada campo:

* *

Deste xeito podemos combinar ambas ferramentas segundo as necesidades de cada parte da interface.

Agora imos aplicar estes conceptos ao noso proxecto.

A intención non é modificar toda a aplicación dunha soa vez. Imos tomar progresivamente elementos que anteriormente estaban definidos mediante CSS e substituílos polas correspondentes clases de Tailwind.

Por exemplo, esta regra:

```
.fila {
   display: flex;
   gap: 1rem;
   width: 100%;
}
```

pode substituírse directamente por:

* *

E unha estrutura de dúas columnas como:

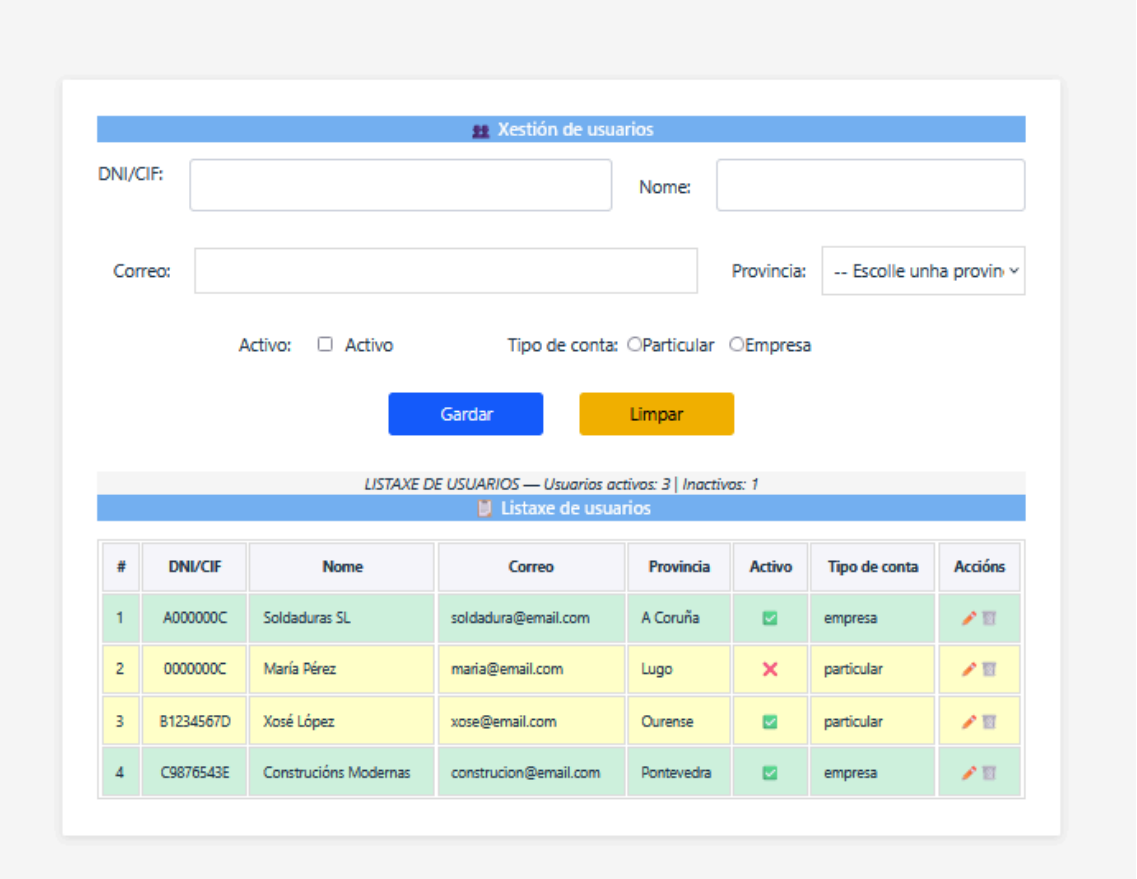
```
.fiorm {
   display: flex;
}
```

pode resolverse todo el *form* mediante:

<form @submit.prevent="gardarUsuario"

class="w-full flex flex-col gap-8 mb-8" >

A medida que avancemos na unidade iremos eliminando progresivamente o CSS específico da aplicación e substituíndoo por utilidades de Tailwind.



A.5. Deseño responsive

Unha aplicación web non se visualiza sempre no mesmo tipo de pantalla. Unha aplicación pensada para un ordenador debe adaptarse tamén a **tablets e dispositivos móbiles**, onde o espazo dispoñible é moito menor.

Ata agora utilizamos clases de Tailwind para definir o aspecto e a distribución dos elementos, pero estas clases **aplícanse, por defecto, independentemente do tamaño da pantalla.**

Tailwind permite modificar o deseño en función do tamaño do dispositivo mediante os seus **modificadores responsive**.

A idea básica é moi sinxela:

class="grid grid-cols-1 md:grid-cols-2"

Neste exemplo:

- *grid-cols-1* → por defecto teremos **unha columna**.
- *md:grid-cols-2* → a partir do tamaño *md*, teremos **dúas columnas**.

Deste xeito podemos facer que **unha mesma interface se comporte de forma diferente segundo o espazo dispoñible.**

A.5.1. Os puntos de ruptura

Tailwind proporciona varios puntos de ruptura predefinidos:

Modificador	Aplicación aproximada
sm:	≥ 640 px
md:	≥ 768 px
lg:	≥ 1024 px
xl:	≥ 1280 px
2xl:	≥ 1536 px

Non debemos interpretar estes valores como tipos concretos de dispositivos. **Non significa que *md* sexa "tablet" ou *lg* sexa "ordenador"**. Son simplemente puntos nos que podemos cambiar o deseño.

Por exemplo:

> * *

Significa:

- Pantallas pequenas → 1 columna.
- A partir de ***md*** → 2 columnas.
- A partir de ***lg*** → 4 columnas.

A.5.2. Deseño mobile-first

Tailwind utiliza un enfoque denominado **mobile-first**.Isto significa que **primeiro definimos o comportamento para pantallas pequenas** e posteriormente engadimos modificacións para pantallas maiores.

Por exemplo:

> * *

Non estamos dicindo:

> "En móbil utiliza unha columna e en tablet dúas".

Estamos dicindo:

> "Por defecto utiliza unha columna e, cando se alcanza ***md***, pasa a utilizar dúas".

Este enfoque facilita a creación de interfaces que funcionan correctamente en dispositivos con pouco espazo.

A.5.3. Aplicación ao formulario

No noso formulario temos actualmente unha estrutura como:

> * *

Isto coloca os elementos nunha fila independentemente do tamaño da pantalla. Nunha pantalla pequena isto pode provocar que os campos queden demasiado estreitos.

Podemos facer que en pantallas pequenas se coloquen verticalmente e que recuperen a distribución horizontal en pantallas maiores:

* *

Agora temos:

- Pantalla pequena:

  DNI/CIF

  [.....]

  Nome

  [.....]

- E a partir de md:

  DNI/CIF [.....] Nome [.....]

Só engadimos dúas clases:

flex-col

md:flex-row

A.5.4. Adaptar o tamaño dos elementos

Os modificadores responsive non serven unicamente para cambiar Flexbox ou Grid. Podemos utilizalos con practicamente calquera utilidade de Tailwind.

Por exemplo:

```
<button class="w-full md:w-1/6">
  Gardar
</button>
```

En pantallas pequenas o botón ocupará todo o ancho:

[Gardar]

Mentres que en pantallas maiores ocupará aproximadamente unha sexta parte do contedor. Tamén podemos modificar o tamaño do texto:

<h4 class="text-lg md:text-xl">

ou o espazado:

* *

Neste último caso teremos menos espazo interior en pantallas pequenas e máis espazo cando exista maior superficie dispoñible.

A.5.5. Aplicación ao noso proxecto

Imos aplicar agora o deseño responsive á primeira fila do formulario.

Podemos utilizar:

* *

Deste xeito non necesitamos manter unha regra CSS específica como:

```
@media (max-width: 768px) {
  .fila {
    flex-direction: column;
    gap: 0.5rem;
  }
}
```

A propia estrutura HTML xa define o comportamento responsive.

Este é precisamente un dos cambios que buscamos ao utilizar Tailwind: **parte das regras CSS que antes tiñamos que escribir manualmente pasan a estar expresadas mediante clases utilitarias.**

A.5.6. Iimportante: non todo debe adaptarse igual

O deseño responsive non consiste simplemente en facer que todos os elementos ocupen o 100 % do ancho en móbil.

Debemos pensar **como queremos que se comporte cada elemento**.

Por exemplo:

- Os campos dun formulario poden pasar dunha fila a unha columna.
- Os botóns poden ocupar todo o ancho en dispositivos pequenos.
- Unha táboa pode necesitar desprazamento horizontal.
- Un menú pode cambiar a súa disposición.
- Os espazos e tamaños de letra poden reducirse.
- Algúns elementos secundarios poden incluso ocultarse.

Por iso, o responsive é antes de nada **unha decisión de deseño**, e Tailwind simplemente nos proporciona as ferramentas para implementala.

Aplicación práctica

Imos revisar agora o formulario da aplicación e adaptalo para que funcione correctamente en pantallas pequenas.

Comezaremos pola primeira fila:

```
<div class="flex flex-col md:flex-row gap-6 w-full">
```

E posteriormente aplicaremos o mesmo criterio ao resto das filas e aos botóns.

O obxectivo non é conseguir simplemente que "non desborde", senón que **a aplicación siga sendo cómoda de utilizar tanto nun ordenador como nun dispositivo móbil**.

A.6. Tipografías: colores, tamaños y sombras

Ata o de agora utilizamos Tailwind principalmente para controlar o **espazado, a distribución dos elementos e o deseño responsive**. Neste apartado imos centrarnos no aspecto visual dos elementos: tipografía, cores, bordes e sombras.

Un dos principais obxectivos de Tailwind é precisamente poder definir estes aspectos directamente mediante clases utilitarias, sen necesidade de crear regras CSS específicas para cada elemento.

A.6.1. Tipografía

Tailwind permite modificar facilmente o tamaño, peso e aliñación do texto.

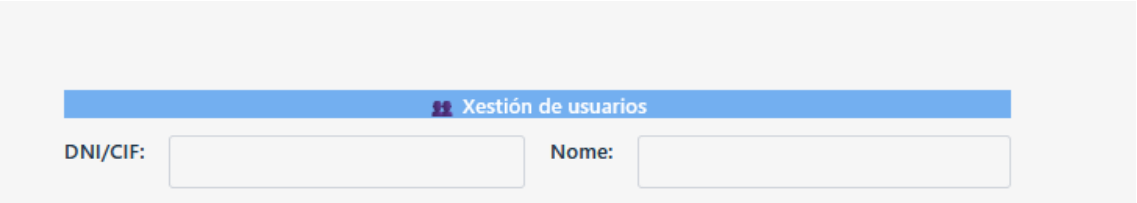
Algunhas das clases máis utilizadas son:

Clase	Función
text-sm	Texto pequeno
text-base	Tamaño normal
text-lg	Texto grande
text-xl	Texto máis grande
text-2xl	Texto grande
font-normal	Peso normal
font-medium	Peso medio
font-semibold	Semi-negra
font-bold	Negra
text-center	Texto centrado
text-left	Aliñado á esquerda
text-right	Aliñado á dereita

Por exemplo:

```
<h4 class="text-xl font-semibold text-center">
  Xestión de usuarios
</h4>
```

Podemos combinar varias utilidades para definir o aspecto completo do texto.



A.6.2. Cores

Tailwind dispón dunha ampla escala de cores. Por exemplo:

```
<p class="text-blue-600">
  Usuario activo
</p>
```

A cor do **texto** establécese mediante ***text-***.

Para o **fondo** utilizamos ***bg-***:

```
<div class="bg-blue-600 text-white">
  Usuarios
</div>
```

As cores dispoñen tamén de diferentes intensidadestal como comentamos :

blue-400

blue-500

blue-600

blue-700

blue-800

Por exemplo:

```
<button class="bg-blue-600 text-white">
  Gardar
</button>
```

ou:

```
<button class="bg-red-600 text-white">
  Eliminar
</button>
```

Isto permite establecer unha linguaxe visual coherente na aplicación.

A.6.3. Bordes

Para engadir un borde podemos utilizar:

```
<input class="border">
```

Tamén podemos modificar a súa **cor**:

```
<input class="border border-gray-300">
```

ou o seu **grosor**:

```
<div class="border-2">
```

Para redondear as esquinas utilizamos rounded:

```
<input class="border border-gray-300 rounded">
```

Podemos aumentar o redondeado:

rounded

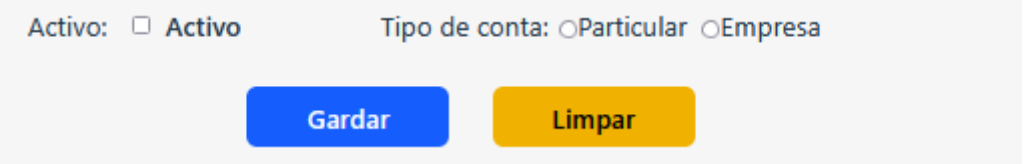
rounded-md

rounded-lg

rounded-xl

Por exemplo:

```
<button class="bg-blue-600 text-white px-4 py-2 rounded-lg">
  Gardar
</button>
```



A.6.4. Sombras

As **sombras** permiten **separar visualmente un elemento do fondo**. As clases máis habituais son:

shadow-sm

shadow

shadow-md

shadow-lg

shadow-xl

Por exemplo:

```
<div class="bg-white p-6 shadow-md rounded-lg">
  Contido
</div>
```

Podemos **combinar varias utilidades para crear unha pequena "tarxeta" ou panel.**

A.6.5. Aplicación ao noso proxecto

No noso proxecto aínda temos algunhas regras CSS que podemos substituír por Tailwind.

Por exemplo, actualmente temos:

```
h4 {
  margin-bottom: 1rem;
  font-weight: 600;
  background-color: #73aff0;
  color: white;
}
```

Podemos levar parte destas propiedades directamente ao HTML:

```
<h4 class="mb-4 font-semibold bg-blue-400 text-white">
  👥 Xestión de usuarios
</h4>
```

Deste xeito:

- *mb-4* → separación inferior.
- *font-semibold* → peso da letra.
- *bg-blue-400* → cor de fondo.
- *text-white* → cor do texto.

Xa non necesitamos manter esas propiedades na regra CSS de h4.

Aplicamos tamén o concepto aos botóns

Os botóns que xa modificamos anteriormente poden quedar, por exemplo:

```
<button
   type="submit"
   class="w-1/6 bg-blue-600 text-white px-4 py-2 rounded-lg"
>
   Gardar
</button>
```

Aquí estamos combinando varias das utilidades estudadas:

w-1/6 → ancho

bg-blue-600 → fondo

text-white → texto

px-4 → espazo horizontal

py-2 → espazo vertical

rounded-lg → esquinas redondeadas

É importante observar que **unha única clase non define o aspecto do botón**. O resultado obtense combinando pequenas utilidades que realizan cada unha unha función concreta.

A.6.6. Non todo debe ser unha clase de Tailwind

Utilizar Tailwind **non significa que debamos eliminar absolutamente todo** o CSS tradicional.

Pode seguir sendo necesario utilizar CSS convencional cando necesitamos:

- comportamentos moi específicos;
- estilos complexos;
- animacións personalizadas;
- regras que se reutilizan en moitos elementos;
- ou características que non resulta práctico expresar mediante clases utilitarias.

O obxectivo é utilizar cada ferramenta cando resulte máis conveniente. No noso proxecto, por exemplo, podemos ir eliminando progresivamente regras CSS sinxelas como:

background-color color

padding margin

border box-shadow

e substituílas polas correspondentes clases de Tailwind.

Isto fará que ao finalizar a unidade teñamos unha aplicación na que **a maior parte do deseño visual está definido mediante Tailwind**, mantendo CSS convencional só cando realmente sexa necesario.

A.7. Estados e interaccións

Ata agora utilizamos Tailwind para definir o aspecto dos elementos nun estado normal. Porén, nunha aplicación web os elementos tamén cambian de aspecto cando o usuario interactúa con eles.

Por exemplo:

- un botón cambia ao pasar o rato por enriba;
- un campo de formulario recibe o foco;
- un botón pode estar desactivado;
- unha ligazón pode cambiar de cor;
- un elemento pode mostrar un aspecto diferente mentres está seleccionado.

Tailwind permite definir estes comportamentos mediante **variantes de estado**.

A.7.1. Estado hover

A variante ***hover:*** permite modificar o aspecto dun elemento cando o usuario sitúa o rato sobre el.

Por exemplo:

```
<button class="bg-blue-600 hover:bg-blue-700 text-white px-4 py-2 rounded-lg">
  Gardar
</button>
```

O botón terá inicialmente:

```
bg-blue-600
```

e cando o usuario pase o rato por enriba:

```
hover:bg-blue-800
```

Non necesitamos escribir unha regra CSS como:

```
button:hover {
  background-color: ...;
}
```

O comportamento queda definido directamente no elemento.



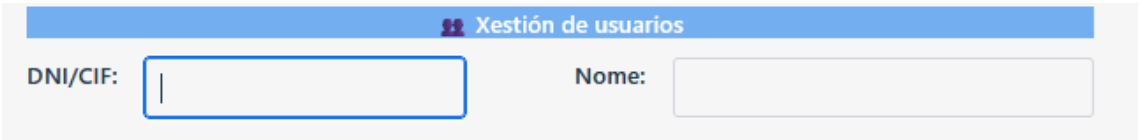


A.7.2. Estado focus

Nos formularios é especialmente importante indicar visualmente cando un campo está seleccionado. Podemos utilizar ***focus***::

```
<input
  type="text"
  class="border border-gray-300 rounded px-3 py-2
      focus:border-blue-500 focus:ring-2 focus:ring-blue-200"
/>
```

Cando o usuario sitúe o cursor no campo, Tailwind aplicará as clases precedidas por ***focus***:. Isto permite proporcionar unha indicación visual clara de **que campo está activo**.



A.7.3. Estado disabled

No noso proxecto xa temos un exemplo especialmente interesante:

```
<button
  :disabled="novoUsuario.dni === '' || novoUsuario.nome === ''" >
  Gardar
</button>
```

Vue determina cando o botón está desactivado. Tailwind permítenos controlar tamén o seu aspecto visual nese estado:

```
<button
  :disabled="novoUsuario.dni === '' || novoUsuario.nome === ""
  class="bg-blue-600 text-white px-4 py-2 rounded-lg
      disabled:bg-gray-400 disabled:cursor-not-allowed"  >
  Gardar
</button>
```

Así:

- normalmente → fondo azul;
- desactivado → fondo gris;
- desactivado → o cursor indica que non se pode utilizar.

Aquí vemos como **Vue e Tailwind traballan conxuntamente**:

Vue
↓
determina o estado do botón

Tailwind
↓
define o aspecto visual dese estado

A.7.4. Combinar estados

As variantes tamén poden combinarse.

Por exemplo:

```
<button
  class="bg-blue-600 hover:bg-blue-700
      focus:ring-2 focus:ring-blue-300
      disabled:bg-gray-400
      text-white px-4 py-2 rounded-lg"
>
  Gardar
</button>
```

Temos agora tres situacións diferentes:

Estado	Aspecto
Normal	Azul
hover	Azul máis escuro
focus	Mostra un indicador de foco
disabled	Gris

Deste xeito podemos definir o comportamento visual completo dun elemento sen escribir regras CSS específicas.

A.7.5. Estados nos campos de formulario

Os estados tamén son especialmente útiles nos ***input***. Por exemplo:

```
<input
  type="email"
  class="w-full border border-gray-300 rounded px-3 py-2
      focus:border-blue-500 focus:ring-2 focus:ring-blue-200"
/>
```

Podemos incluso utilizar os estados de validación:

```
<input
  type="email"
  class="border border-gray-300
      invalid:border-red-500
      focus:invalid:ring-2"
/>
```



Neste caso o aspecto do campo pode cambiar cando o navegador considera que o valor introducido non é válido.

A.7.6. Aplicación ao noso proxecto

Podemos aplicar estes estados aos botóns de **Gardar** e **Limpar**.

Por exemplo:

```
<button
  type="submit"
  class="w-1/6 bg-blue-600 hover:bg-blue-700
      text-white px-4 py-2 rounded-lg
      disabled:bg-gray-400 disabled:cursor-not-allowed"
  :disabled="novoUsuario.dni === '' || novoUsuario.nome === ''"
>
  Gardar
</button>
```

E para o botón de limpar:

```
<button
  type="button"
  class="w-1/6 bg-yellow-500 hover:bg-yellow-600
      text-black px-4 py-2 rounded-lg"
  @click="limparForm"
>
  Limpar
</button>
```

Agora os botóns proporcionan unha resposta visual cando o usuario interactúa con eles.

Importante: As variantes de estado de Tailwind **non substitúen a lóxica da aplicación**.

Por exemplo:

```
disabled:bg-gray-400
```

non desactiva realmente o botón.

Simplemente indica:

"Se o elemento está desactivado, utiliza este estilo".

É Vue, mediante:

```
:disabled="..."
```

quen determina se o botón está ou non desactivado.Polo tanto, volvemos atoparnos coa combinación de ferramentas:

Vue controla o comportamento → Tailwind controla a presentación.

Esta separación será especialmente importante a medida que a aplicación vaia incorporando funcionalidades máis complexas.

A.7. Estados e interaccións

Os formularios e as táboas son dous elementos fundamentais nas aplicacións de xestión, xa que permiten **introducir, modificar e visualizar información**.

Na nosa aplicación xa contamos con ambos elementos: o formulario de xestión de usuarios e a táboa na que se mostra a listaxe. Neste apartado imos adaptar o seu deseño utilizando as clases utilitarias de Tailwind.

O obxectivo non é aprender novas funcionalidades de Vue, senón comprobar como podemos **deseñar elementos habituais dunha aplicación web utilizando Tailwind CSS**.

A.8.1. Deseño de formularios

Tailwind permite aplicar directamente clases aos diferentes elementos dun formulario***: input, select, label, textarea e button.***

Por exemplo, un campo de texto pode definirse así:

```
<div class="flex flex-col gap-2">
  <label class="font-medium">Nome:</label>
  <input
    type="text"
    class="w-full px-3 py-2.5 border border-gray-300 rounded-md
        focus:border-blue-500 focus:ring-2 focus:ring-blue-200"
  />
</div>
```

Aquí estamos combinando varias utilidades:

- ***flex flex-col*** → coloca o label e o input en columna.
- ***gap-2*** → separa ambos elementos.
- ***font-medium*** → dá máis peso ao texto do label.
- ***w-full*** → o campo ocupa todo o ancho dispoñible.
- ***px-3 py-2.5*** → establece espazo interior.

- ***border border-gray-300*** → define o borde.
- ***rounded-md*** → redondea as esquinas.
- ***focus:border-blue-500*** → cambia a cor do borde ao seleccionar o campo.
- ***focus:ring-2*** → mostra un indicador visual de foco.

Deste xeito podemos crear un campo completo sen necesidade de definir unha clase CSS específica.

A.8.2. Organización dos campos

Un formulario adoita conter varios campos e debemos organizar a súa distribución. Podemos utilizar Grid:

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-6">
  <div class="flex flex-col gap-2">
    <label>DNI/CIF:</label>
    <input type="text" class="border rounded-md px-3 py-2.5">
  </div>
  <div class="flex flex-col gap-2">
    <label>Nome:</label>
    <input type="text" class="border rounded-md px-3 py-2.5">
  </div>
  <div class="flex flex-col gap-2">
    <label>Correo:</label>
    <input type="email" class="border rounded-md px-3 py-2.5">
  </div>
  <div class="flex flex-col gap-2">
    <label>Provincia:</label>
    <select class="border rounded-md px-3 py-2.5">
      <option>A Coruña</option>
      <option>Lugo</option>
```

```
            <option>Ourense</option>
            <option>Pontevedra</option>
          </select>
        </div>
      </div>
```

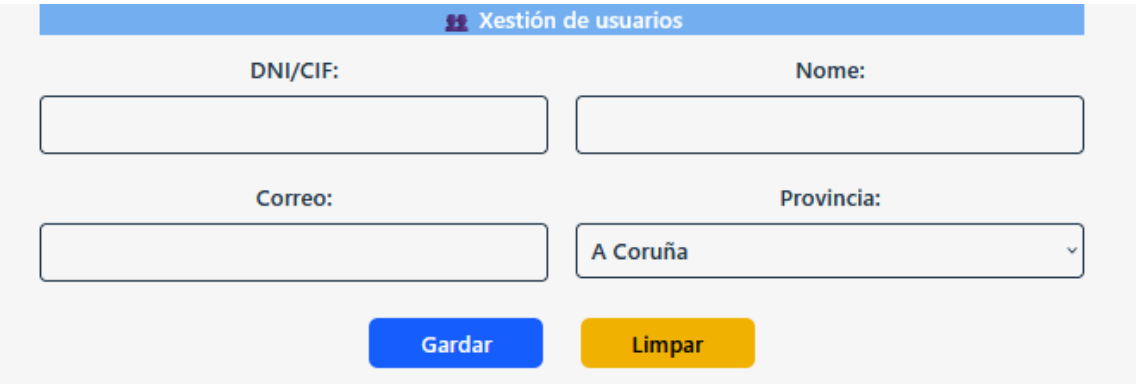
A combinación:

```
grid-cols-1 md:grid-cols-2
```

fai que os campos aparezan:

- **nunha columna** en pantallas pequenas;
- en **dúas columnas** a partir de md.

Aquí estamos combinando os conceptos estudados nos apartados anteriores: **Grid, espazado e responsive**.



A.8.3. Select e outros controis

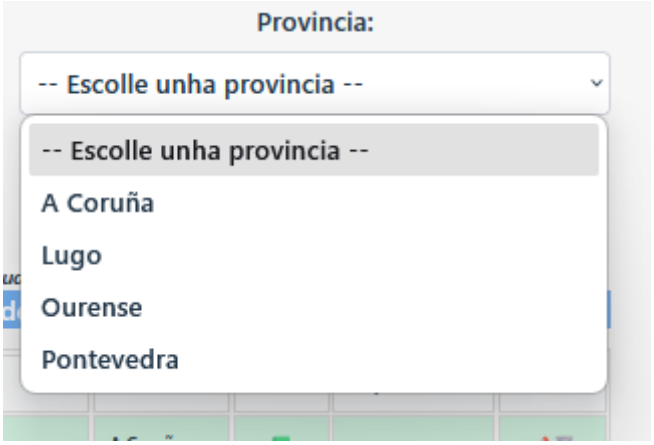
Os elementos select poden recibir as mesmas utilidades que os input.

Por exemplo:

```
<select
  class="w-full px-3 py-2.5 border border-gray-300
      rounded-md bg-white"
>
```

```
        <option value="">-- Escolle unha provincia --</option>
        <option>A Coruña</option>
        <option>Lugo</option>
        <option>Ourense</option>
        <option>Pontevedra</option>
    </select>
```

O importante é manter unha presentación coherente entre os diferentes controis do formulario.



A.8.4. Táboas

As táboas son especialmente útiles nas aplicacións de xestión para mostrar información procedente do backend. Tailwind permite definir directamente o aspecto da táboa.

Unha estrutura básica sería:

```
<div class="overflow-x-auto">
  <table class="w-full border-collapse">

    <thead>
      <tr class="bg-gray-100">
        <th class="border px-4 py-3 text-center">
          Nome
        </th>

        <th class="border px-4 py-3 text-center">
```

```
            Correo
          </th>

          <th class="border px-4 py-3 text-center">
            Provincia
          </th>
        </tr>
      </thead>

      <tbody>
        <tr>
          <td class="border px-4 py-3">
            María Pérez
          </td>

          <td class="border px-4 py-3">
            maria@email.com
          </td>

          <td class="border px-4 py-3">
            Lugo
          </td>
        </tr>
      </tbody>

    </table>
  </div>
```

Algunhas clases importantes son:

Clase	Función
w-full	Ocupa todo o ancho
border-collapse	Une os bordes das celas
border	Engade borde
px-4	Espazo horizontal
py-3	Espazo vertical
text-center	Centra o contido
bg-gray-100	Fondo gris claro

A.8.5. Táboas con moitos datos

Nun dispositivo móbil unha táboa pode ser demasiado ancha para a pantalla.Para evitar que rompa o deseño podemos utilizar:

```
<div class="overflow-x-auto">
  <table class="w-full">
    ...
  </table>
</div>
```

overflow-x-auto permite desprazar horizontalmente a táboa cando o seu ancho supera o espazo dispoñible. Isto é especialmente importante nas aplicacións de xestión, onde unha táboa pode conter moitas columnas.

No noso proxecto temos precisamente este problema potencial, xa que a táboa de usuarios contén:

| DNI/CIF | Nome | Correo | Provincia | Activo | Tipo de conta | Accións

En pantallas pequenas non debemos intentar reducir indefinidamente o tamaño das columnas. En determinados casos é máis práctico **permitir o desprazamento horizontal.**

A.8.6. Filas e estados visuais

Tamén podemos utilizar Tailwind para destacar visualmente determinadas filas.

Por exemplo:

```
<tr class="bg-green-50">
```

ou:

```
<tr class="bg-yellow-50">
```

Na nosa aplicación xa utilizabamos CSS para diferenciar usuarios segundo o tipo de conta:

```
.fila-empresa {
  background-color: #d0f0df;
}

.fila-particular {
  background-color: #ffffcc;
}
```

Isto podería expresarse directamente mediante clases de Tailwind se coñecemos o estado no momento de renderizar a fila.

Neste caso, con todo, **a clase depende dun dato que procede de Vue**, polo que será especialmente interesante no seguinte apartado.

A.8.7. Formularios e Vue

É importante distinguir entre o deseño do formulario e o seu funcionamento. Tailwind encárgase da **presentación**:

```
class="border rounded-md px-3 py-2.5"
```

Mentres que Vue continúa encargándose da **lóxica**:

```
v-model="novoUsuario.nome"
```

Por exemplo:

```
<input
  v-model="novoUsuario.nome"
  type="text"
```

```
    class="w-full px-3 py-2.5 border border-gray-300 rounded-md"
/>
```

Neste cas**o:**

Tailwind → como se ve o campo Vue → que dato contén o campo

Isto permite manter separadas as responsabilidades de cada ferramenta.

A.8.8. Aplicación ao noso proxecto

Neste punto xa podemos comezar a substituír progresivamente o CSS que utilizabamos para o formulario e a táboa de XestionUsuarios.vue.

Por exemplo, unha regra como:

```
.campo input {
  padding: 0.5rem;
  border: 1px solid #ddd;
  border-radius: 0;
}
```

pode substituírse por:

```
<input
  class="w-full px-3 py-2.5 border border-gray-300 rounded-md"
>
```

E unha regra da táboa como:

```
th,
td {
  border: 1px solid #ddd;
  padding: 0.7rem;
}
```

pode trasladarse directamente aos elementos:

<th class="border px-4 py-3">

<td class="border px-4 py-3">

Deste xeito imos reducindo progresivamente o CSS específico da aplicación.

Non se trata de eliminar CSS por principio, senón de comprobar que moitas das regras sinxelas que necesitabamos anteriormente xa poden expresarse directamente mediante Tailwind.

No seguinte apartado veremos un caso especialmente importante: **cando a clase que queremos aplicar non é fixa, senón que depende dos datos ou do estado da aplicación Vue.**

A.9. Clases dinámicas e integración con Vue

Nos apartados anteriores utilizamos clases de Tailwind directamente no HTML:

```
<button class="bg-blue-600 text-white px-4 py-2 rounded-lg">
  Gardar
</button>
```

Neste caso, as clases son **fixas**. O botón sempre terá o mesmo aspecto. Nas aplicacións desenvolvidas con Vue, porén, é moi habitual que o **aspecto dun elemento dependa dos datos ou do estado da aplicación.**

Por exemplo, na nosa aplicación:

- unha conta de tipo **empresa** pode aparecer cun fondo verde;
- unha conta **particular** pode aparecer cun fondo amarelo;
- un usuario inactivo pode mostrarse de forma diferente;
- un botón pode estar desactivado dependendo dos datos introducidos.

Neste tipo de situacións necesitamos combinar **Vue e Tailwind**.

A.9.1. A directiva *:class*

Vue permite modificar as clases dun elemento mediante a directiva ***:class.***

Por exemplo:

```
<p :class="usuario.activo ? 'text-green-600' : 'text-red-600'">
  {{ usuario.activo ? 'Activo' : 'Inactivo' }}
</p>
```

Se *usuario.activo* é *true*, aplícase:

```
text-green-600
```

Se é *false*:

```
text-red-600
```

Aquí Tailwind define **como se representa visualmente cada estado**, mentres que Vue decide **que estado corresponde aplicar**.

A.9.2. Clases dinámicas con obxectos

Outra forma habitual de utilizar `:class` é mediante un obxecto:

```
<div
  :class="{
    'bg-green-100': usuario.activo,
    'bg-gray-100': !usuario.activo
  }"  >
  {{ usuario.nome }}
</div>
```

Vue comprobará as condicións e aplicará a clase correspondente. Este formato resulta especialmente útil cando temos máis dun estado.

A.9.3. Aplicación ao noso proxecto

Na nosa aplicación xa temos unha función:

```
function tipoUsuario(tipo) {
  if (tipo === "empresa")
    return "fila-empresa"

  if (tipo === "particular")
    return "fila-particular"

  return ""
}
```

E no template:

```
<tr
  v-for="(u, index) in usuarios"
  :key="index"
  :class="tipoUsuario(u.tipoCuenta)"
>
```

Ata agora as clases estaban definidas no CSS:

```
.fila-empresa {
  background-color: #d0f0df;
}
.fila-particular {
  background-color: #ffffcc;
}
```

Podemos eliminar esas regras CSS e **utilizar directamente clases de Tailwind**. Por exemplo:

```
<tr
  v-for="(u, index) in usuarios"
  :key="index"
  :class="{
     'bg-green-100': u.tipoCuenta === 'empresa',
     'bg-yellow-100': u.tipoCuenta === 'particular'
  }"
>
```

Agora o funcionamento é:

tipoCuenta = empresa ⇒ bg-green-100

tipoCuenta = particular ⇒ bg-yellow-100

Xa non necesitamos:

~~.fila-empresa { ... }~~

~~.fila-particular { ... }~~

A.9.4. Clases fixas e dinámicas na mesma etiqueta

Podemos combinar clases que sempre estarán presentes con outras que dependerán dos datos.

Por exemplo:

```
<tr
  v-for="(u, index) in usuarios"
  :key="index"
  class="hover:bg-gray-50"
  :class="{
     'bg-green-100': u.tipoCuenta === 'empresa',
     'bg-yellow-100': u.tipoCuenta === 'particular'
  }"
>
```

Neste caso:

class="hover:bg-gray-50"

é unha clase fixa.

Mentres que:

:class="{ ... }"

contén clases dinámicas.

Así podemos ter:

- aspecto común para todas as filas;
- aspecto específico segundo o tipo de usuario;
- interacción visual ao pasar o rato.

A.9.5. Clases dinámicas e estado

As clases dinámicas tamén poden depender de propiedades booleanas.

Por exemplo, podemos destacar os usuarios activos:

```
<td
  :class="u.activo ? 'text-green-600 font-semibold' : 'text-gray-400'"
>
  {{ u.activo ? 'Activo' : 'Inactivo' }}
</td>
```

O resultado sería:

Usuario activo ⇒ texto verde e semi-negra

Usuario inactivo ⇒ texto gris

Podemos combinar varias clases dentro da mesma condición:

```
:class="u.activo
  ? 'text-green-600 font-semibold'
  : 'text-gray-400'"
```

A.9.6. Clases dinámicas nos formularios

Este mecanismo tamén resulta útil nos formularios.

Por exemplo, podemos modificar a cor do borde do campo `Nome` segundo o tipo de conta seleccionado:

```
<input
  v-model="novoUsuario.nome"
  type="text"
  :class="{
     'border-green-500': novoUsuario.tipoCuenta === 'empresa',
     'border-yellow-500': novoUsuario.tipoCuenta === 'particular',
     'border-gray-300': novoUsuario.tipoCuenta === ''
  }"
/>
```

Agora o borde do campo cambia automaticamente cando o usuario selecciona o tipo de conta.

O proceso é:

Selecciona "Empresa" ⇒ novoUsuario.tipoCuenta = "empresa"

Vue actualiza a clase e Tailwind aplica border-green-500

Non necesitamos manipular manualmente o DOM.

A.9.7. Integración entre Vue e Tailwind

Este é un dos puntos máis importantes desta unidade. **Vue e Tailwind non compiten entre si.**

Cada ferramenta ten unha responsabilidade diferente.

Por exemplo:

Vue	Tailwind
Datos	Presentación
Estado	Cores
Lóxica	Espazado
Eventos	Tamaños
Reactividade	Bordes
Condicións	Sombras
Clases dinámicas	Estados visuais

```
<tr
  :class="{
    'bg-green-100': u.tipoCuenta === 'empresa',
    'bg-yellow-100': u.tipoCuenta === 'particular'
  }"
>
```

Vue decide:

"Este usuario é unha empresa."

Tailwind decide:

"Para unha empresa utilizaremos este fondo verde."

A.9.8. Unha recomendación importante

Cando traballamos con Tailwind e Vue **debemos evitar construír clases mediante concatenación arbitraria de cadeas sempre que sexa posible.**

Por exemplo, non é recomendable facer:

```
const cor = "green"
```

e despois:

```
<div :class="'bg-' + cor + '-500'">
```

Tailwind necesita detectar as clases que debe xerar durante a compilación e este tipo de construcións dinámicas poden provocar que determinadas clases non sexan xeradas. É preferible utilizar clases completas:

```
<div
  :class="{
     'bg-green-500': cor === 'green',
     'bg-red-500': cor === 'red'
  }"
>
```

Deste xeito as clases aparecen explicitamente no código e Tailwind pode procesalas correctamente.

A.9.9. Aplicación final ao noso listado

Podemos deixar a fila da nosa táboa deste xeito:

```
<tr
  v-for="(u, index) in usuarios"
  :key="index"
  class="hover:bg-gray-50"
  :class="{
     'bg-green-100': u.tipoCuenta === 'empresa',
     'bg-yellow-100': u.tipoCuenta === 'particular'
  }"
>
```

Con isto conseguimos, utilizando unicamente Tailwind e Vue:

- diferenciar empresas e particulares;
- manter un efecto `hover`;
- eliminar as clases CSS `.fila-empresa` e `.fila-particular`;
- facer que o aspecto dependa dos datos da aplicación.

Este é o punto no que **Tailwind deixa de ser simplemente unha ferramenta para maquillar a interface e pasa a integrarse directamente coa lóxica reactiva de Vue**.

A partir de aquí xa podemos dar un paso máis: **extraer partes da interface e convertelas en compoñentes reutilizables**, que será o obxectivo do seguinte apartado.

A.10. Creación e reutilización de compoñentes

Unha das principais vantaxes de Vue é a posibilidade de dividir a interface dunha aplicación en **compoñentes independentes e reutilizables**. Nunha aplicación pequena podemos ter todo nunha única vista, pero **a medida que aumenta a súa complexidade resulta máis conveniente dividir a interface** en partes máis pequenas.

Por exemplo, na nosa aplicación xa temos compoñentes como:

App.vue

├── ***NavBar.vue***

├── ***XestionUsuarios.vue***

└── ***XestionTarefas.vue***

Cada compoñente encárgase dunha parte concreta da interface. T**ailwind encaixa especialmente ben con esta forma de traballar,** xa que os estilos poden definirse directamente dentro do propio compoñente.

A.10.1. Que é un compoñente reutilizable?

Un **compoñente reutilizable** é unha parte da interface que **podemos utilizar en diferentes lugares** da aplicación sen ter que repetir o código. Por exemplo, se a aplicación ten varios botóns de acción podemos crear un compoñente:

BotonAccion.vue

e utilizalo posteriormente:

```
<BotonAccion>
  Gardar
</BotonAccion>
```

ou:

```
<BotonAccion>
  Limpar
</BotonAccion>
```

O obxectivo é evitar repetir continuamente a mesma estrutura HTML e as mesmas clases de Tailwind.

A.10.2. Un compoñente sinxelo

Podemos crear un compoñente chamado ***BotonAccion.vue:***

```
<template>
  <button
    class="px-4 py-2 rounded-lg bg-blue-600 text-white
        hover:bg-blue-700"
  >
    <slot></slot>
  </button>
</template>
```

O elemento:

```
<slot></slot>
```

indica onde se colocará o contido que escribamos ao utilizar o compoñente.

Por exemplo:

```
<BotonAccion>
  Gardar
</BotonAccion>
```

producirá visualmente: Gardar

A.10.3. Por que reutilizar compoñentes?

Se temos dez botóns cunha presentación similar, non ten sentido repetir en todos eles:

px-4

py-2

rounded-lg

bg-blue-600

text-white

hover:bg-blue-700

Un compoñente permítenos centralizar esa definición. Se posteriormente queremos cambiar:

rounded-lg

por:

rounded-md

podemos facelo no compoñente e o cambio afectará a todos os lugares onde se utiliza.

Isto facilita:

- a reutilización;
- o mantemento;
- a coherencia visual;
- a modificación posterior da interface.

A.10.4. Props e personalización

Un compoñente reutilizable normalmente necesita recibir información do exterior.

Vue permite facelo mediante **props**.

Por exemplo:

```
<script setup>
defineProps({
  texto: String
})
</script>

<template>
  <button
    class="px-4 py-2 rounded-lg bg-blue-600 text-white
        hover:bg-blue-700"
  >
    {{ texto }}
  </button>
</template>
```

Agora podemos utilizar: *<BotonAccion texto="Gardar" />*

ou: *<BotonAccion texto="Continuar" />*

O mesmo compoñente pode mostrar textos diferentes.

A.10.5. Variantes visuais

Podemos ir un paso máis alá e facer que o aspecto do botón dependa dunha prop. Por exemplo:

```
<script setup>
const props = defineProps({
  texto: String,
  tipo: {
    type: String,
```

```
            default: 'primary'
        }
    })
</script>

<template>
  <button
    :class="{
      'bg-blue-600 hover:bg-blue-700 text-white':
        props.tipo === 'primary',

      'bg-yellow-500 hover:bg-yellow-600 text-black':
        props.tipo === 'warning',

      'bg-red-600 hover:bg-red-700 text-white':
        props.tipo === 'danger'
    }"
    class="px-4 py-2 rounded-lg"
  >
    {{ props.texto }}
  </button>
</template>
```

Agora podemos utilizar o mesmo compoñente con diferentes aparencias:

```
<BotonAccion texto="Gardar" tipo="primary" />

<BotonAccion texto="Limpar" tipo="warning" />

<BotonAccion texto="Eliminar" tipo="danger" />
```

Aquí estamos aplicando, unha vez máis, directamente o aprendido no apartado anterior:

Vue determina a clase → Tailwind determina o aspecto.

A.10.6. O uso de slot

As props son apropiadas cando queremos pasar datos concretos ao compoñente. Os **slots**, en cambio, son especialmente útiles cando **queremos introducir contido HTML** dentro do compoñente. Por exemplo:

```
<template>
  <button
    class="px-4 py-2 rounded-lg bg-blue-600
        hover:bg-blue-700 text-white"
  >
    <slot></slot>
  </button>
</template>
```

Podemos utilizalo así:

```
<BotonAccion>
  💾 Gardar
</BotonAccion>
```

ou:

```
<BotonAccion>
  🗑️ Eliminar
</BotonAccion>
```

Isto proporciona unha maior flexibilidade.

A.10.7. Compoñentes e Tailwind

Unha das características interesantes de Tailwind é que os estilos quedan moi próximos ao elemento ao que se aplican. Por exemplo:

```
<template>
  <div class="bg-white rounded-lg shadow-md p-6">
    <h3 class="text-xl font-semibold mb-4">
      Información do usuario
    </h3>

    <p class="text-gray-600">
      Datos do usuario
    </p>
  </div>
</template>
```

Este compoñente xa contén tanto:

- a estrutura HTML;
- como o seu deseño visual.

Non necesitamos crear unha folla CSS específica para definir:

```
.tarxeta { ... }

.titulo { ... }

.descripcion { ... }
```

A.10.8. Cando crear un compoñente?

Non debemos converter automaticamente cada elemento HTML nun compoñente.

Ten sentido crear un compoñente cando:

- unha parte da interface se repite;
- ten unha responsabilidade concreta;
- posúe certa complexidade;
- queremos reutilizala en varias vistas;
- ou queremos illar unha parte da aplicación para facilitar o seu mantemento.

Por exemplo, na nosa aplicación poderían ser bos candidatos:

- NavBar.vue
- BotonAccion.vue
- FormularioUsuario.vue
- FormularioTarefa.vue
- TabelaUsuarios.vue
- TabelaTarefas.vue

Non sería necesario crear, por exemplo, un compoñente simplemente para representar un div que só aparece unha vez e non ten comportamento propio.

A nosa aplicación xa utiliza unha estrutura baseada en compoñentes e o que estamos facendo nesta unidade é avanzar cara a unha estrutura máis organizada, na que os compoñentes non só separen funcionalmente a aplicación, senón que tamén permitan **reutilizar elementos visuais e estilos**.

A.10.10. Unha consideración importante

A reutilización non consiste simplemente en evitar escribir código dúas veces. Tamén busca conseguir que a aplicación teña unha **linguaxe visual coherente**.

Se todos os botóns principais da aplicación utilizan:

bg-blue-600 hover:bg-blue-700 rounded-lg

o usuario percibirá que todos pertencen á mesma aplicación.

Se máis adiante decidimos modificar o deseño, **poderemos facelo nos compoñentes correspondentes** en lugar de revisar manualmente todas as vistas.

Por iso, a combinación:

Vue + compoñentes + Tailwind

permite construír interfaces máis **modulares, coherentes e fáciles de manter**.

No seguinte apartado aplicaremos todo o aprendido ata agora ao noso proxecto: faremos o **redeseño da aplicación de xestión**, substituíndo progresivamente o CSS tradicional polas utilidades de Tailwind.

A.11. Rediseño da aplicación

Chegados a este punto xa coñecemos as principais ferramentas de Tailwind que necesitamos para modificar a interface da nosa aplicación. Neste apartado imos **aplicar de forma práctica todo o aprendido** sobre o proxecto de xestión de usuarios e tarefas. O obxectivo non é engadir novas funcionalidades, senón **mellorar e modernizar a interface existente utilizando Tailwind CSS**.

A.11.1. Obxectivo do redeseño

A aplicación foi desenvolvida inicialmente utilizando CSS tradicional. Agora queremos conseguir que:

- a interface teña un aspecto máis moderno;
- o deseño sexa coherente en todas as vistas;
- se adapte a diferentes tamaños de pantalla;
- se reduza o CSS específico da aplicación;
- se utilicen as clases utilitarias de Tailwind;
- e se manteña toda a funcionalidade existente de Vue.

É importante entender que **redeseñar non significa volver desenvolver a aplicación**. A lóxica debe seguir funcionando exactamente igual.

ANTES: Vue ⇒ HTML + CSS tradicional

DESPOIS: Vue ⇒ HTML + Tailwind CSS

A.11.2. Primeiro paso: analizar o CSS existente

Antes de eliminar CSS debemos revisar que función cumpre cada regra.

Por exemplo, podemos atopar:

```
.campo input {
  padding: 0.5rem;
  border: 1px solid #ddd;
  border-radius: 0;
}
```

Moitas destas propiedades poden substituírse directamente por Tailwind:

```
<input class="w-full px-3 py-2 border border-gray-300 rounded">
```

Pero non debemos eliminar unha regra simplemente porque esteamos utilizando Tailwind. Debemos comprobar primeiro se existe unha utilidade de Tailwind que substitúa correctamente esta regra. Se a resposta é si, podemos trasladala ao HTML.

A.11.3. Redeseño do formulario

Comezaremos polo formulario de usuarios. Unha posible estrutura utilizando Grid sería:

```
<form
  @submit.prevent="gardarUsuario"
  class="flex flex-col gap-6"
>

  <div class="grid grid-cols-1 md:grid-cols-2 gap-6">

    <div class="flex flex-col gap-2">
      <label class="font-medium">
        DNI/CIF:
```

```
    </label>

    <input
      v-model="novoUsuario.dni"
      type="text"
      required
      class="w-full px-3 py-2.5
          border border-gray-300
          rounded-md
          focus:border-blue-500
          focus:ring-2 focus:ring-blue-200"
    />
  </div>

  <div class="flex flex-col gap-2">
    <label class="font-medium">
      Nome:
    </label>

    <input
      v-model="novoUsuario.nome"
      type="text"
      required
      class="w-full px-3 py-2.5
          border border-gray-300
          rounded-md
          focus:border-blue-500
          focus:ring-2 focus:ring-blue-200"
```

```
        />
      </div>

    </div>
  </form>
```

Aquí estamos aplicando simultaneamente:

- **Flexbox;**
- **Grid;**
- **espazado;**
- **responsive;**
- **tipografía;**
- **bordes;**
- **estados de foco.**

A.11.4. Redeseño dos botóns

Os botóns tamén deben seguir unha linguaxe visual común. Por exemplo:

```
<div class="flex justify-center gap-8">
  <button
    type="submit"
    class="w-1/6 bg-blue-600
        hover:bg-blue-700
        text-white
        px-4 py-2
        rounded-lg
        disabled:bg-gray-400
        disabled:cursor-not-allowed"
```

```
    >
      Gardar
    </button>

    <button
      type="button"
      class="w-1/6 bg-yellow-500
          hover:bg-yellow-600
          text-black
          px-4 py-2
          rounded-lg"
      @click="limparForm"
    >
      Limpar
    </button>

  </div>
```

Podemos observar que o comportamento dos botóns continúa dependendo de Vue:

```
@click="limparForm"
```

e:

```
:disabled="..."
```

Mentres que Tailwind se ocupa da súa presentación.

A.11.5. Redeseño da táboa

A táboa tamén pode ser modernizada:

```
<div class="overflow-x-auto">

  <table class="w-full border-collapse text-sm">

    <thead>
      <tr class="bg-gray-100">
        <th class="border px-4 py-3 text-center">
          #
        </th>

        <th class="border px-4 py-3 text-center">
          DNI/CIF
        </th>

        <th class="border px-4 py-3 text-center">
          Nome
        </th>

        <th class="border px-4 py-3 text-center">
          Correo
        </th>
      </tr>
    </thead>

    <tbody>
```

```
        ...
      </tbody>

    </table>

  </div>
```

E as celas:

```
  <td class="border px-4 py-3">
    {{ u.nome }}
  </td>
```

Xa non necesitamos definir no CSS:

```
th,
td {
  border: 1px solid #ddd;
  padding: 0.7rem;
}
```

As propiedades están directamente nas clases.

A.11.6. Aplicar clases dinámicas

Agora podemos utilizar o aprendido no apartado A.9 para manter a diferenciación entre tipos de usuario.

```
  <tr
    v-for="(u, index) in usuarios"
    :key="index"
```

```
  class="hover:bg-gray-50"

  :class="{
    'bg-green-100': u.tipoCuenta === 'empresa',
    'bg-yellow-100': u.tipoCuenta === 'particular'
  }"
>
```

Deste xeito eliminamos:

```
.fila-empresa {
  background-color: #d0f0df;
}

.fila-particular {
  background-color: #ffffcc;
}
```

E mantemos o mesmo comportamento visual utilizando Tailwind.

A.11.7. Contedor principal

Tamén podemos mellorar o contedor da aplicación.Por exemplo:

```
<div
  class="w-full bg-white
      p-6 md:p-8
      rounded-lg
      shadow-md"
>
```

Aquí temos:

w-full → ancho completo

bg-white → fondo branco

p-6 → espazado interior

md:p-8 → máis espazo en pantallas grandes

rounded-lg → esquinas redondeadas

shadow-md → sombra

A aplicación comeza a adquirir unha apariencia máis próxima á dunha interface moderna.

A.11.8. Eliminar progresivamente o CSS

Unha vez trasladados os estilos ao HTML podemos revisar o *<style scoped>*. Por exemplo, se temos:

```
h4 {
  margin-bottom: 1rem;
  font-weight: 600;
  background-color: #73aff0;
  color: white;
}
```

e xa substituímos o elemento por:

```
<h4 class="mb-4 font-semibold bg-blue-400 text-white">
  👥 Xestión de usuarios
</h4>
```

podemos eliminar a regra CSS correspondente.

O mesmo podemos facer con:

- **padding;**
- **margin;**
- **background-color;**
- **color;**
- **border;**
- **border-radius;**
- **box-shadow;**
- **font-size;**
- **font-weight;**
- **regras sinxelas de Flexbox e Grid.**

A.11.9. Que CSS podemos manter?

Non é necesario que o resultado final teña **cero CSS tradicional**. Podemos manter regras cando sexan realmente necesarias. Por exemplo, pode ser razoable conservar CSS para:

- **animacións específicas;**
- **estilos moi particulares;**
- **personalizacións que non teñan unha utilidade directa en Tailwind;**
- **regras complexas;**
- **características específicas dun compoñen**te.

O importante é evitar manter CSS duplicado. Se temos:

padding: 0.5rem;

e xa temos:

class="p-2"

non ten sentido conservar ambas regras.

A.11.10. Comprobar que a funcionalidade non se rompe

O redeseño debe ser exclusivamente visual. Despois de modificar un compoñente debemos comprobar que continúa funcionando:

Usuarios

- Crear usuario.
- Editar usuario.
- Eliminar usuario.
- Limpar formulario.
- Activar/desactivar usuario.
- Seleccionar tipo de conta.
- Mostrar correctamente a listaxe.

Tarefas

- Crear tarefa.
- Editar tarefa.
- Eliminar tarefa.
- Comprobar os filtros ou estados existentes.

Navegación

- Cambiar entre as diferentes vistas.
- Comprobar as rutas.
- Comprobar que os compoñentes se cargan correctamente.

O principio fundamental é:

> **Podemos cambiar completamente a presentación sen modificar a lóxica da aplicación.**

A.11.11. Comprobación responsive

Finalmente debemos probar a aplicación con diferentes tamaños de pantalla.

En particular:

- **Móbil**
- **Portátil**
- **Monitor grande**

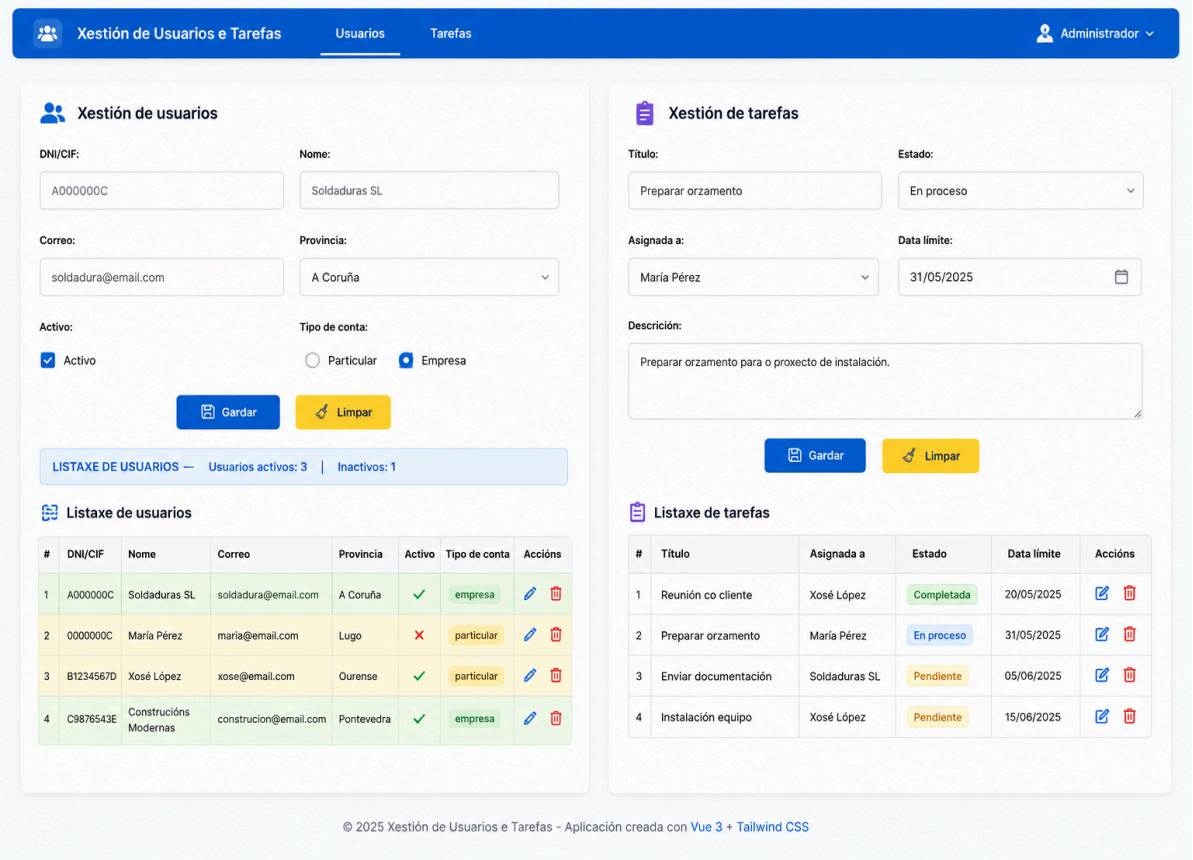
No navegador podemos utilizar as ferramentas de desenvolvemento e modificar o ancho da xanela. Debemos comprobar especialmente:

- **formulario;**
- **botóns;**
- **táboas;**
- **navegación;**
- **espazado;**
- **tamaño dos textos;**
- **aparición do scroll horizontal nas táboas cando sexa necesario.**

A.11.12. Resultado final

Ao finalizar o redeseño teremos unha aplicación na que o máis importante é que **non estamos simplemente cambiando cores ou botóns**. Estamos aprendendo a **estruturar unha interface moderna utilizando un framework CSS utilitario** e integrándoo coa arquitectura de compoñentes de Vue.

Este redeseño será tamén unha boa base para introducir posteriormente novas funcionalidades na aplicación, xa que teremos **unha interface máis coherente e un código visual máis fácil de manter.**


Xestión de Usuarios e Tarefas
Usuarios
Tarefas
Administrador
Xestión de usuarios
DNI/CIF:
A000000C
Nome:
Soldaduras SL
Correo:
soldadura@email.com
Provincia:
A Coruña
Activo:
Activo
Tipo de conta:
Particular
Empresa
Gardar
Limpar
LISTAXE DE USUARIOS — Usuarios activos: 3 | Inactivos: 1
Listaxe de usuarios
DNI/CIF Nome Correo Provincia Activo Tipo de conta Accións
1 A000000C Soldaduras SL soldadura@email.com A Coruña empresa
2 0000000C María Pérez maria@email.com Lugo particular
3 B1234567D Xosé López xose@email.com Ourense particular
4 C9876543E Construcións Modernas construcion@email.com Pontevedra empresa
Xestión de tarefas
Título:
Preparar orzamento
Estado:
En proceso
Asignada a:
María Pérez
Data límite:
31/05/2025
Descrición:
Preparar orzamento para o proxecto de instalación.
Gardar
Limpar
Listaxe de tarefas
Título Asignada a Estado Data límite Accións
1 Reunión co cliente Xosé López Completada 20/05/2025
2 Preparar orzamento María Pérez En proceso 31/05/2025
3 Enviar documentación Soldaduras SL Pendiente 05/06/2025
4 Instalación equipo Xosé López Pendiente 15/06/2025
© 2025 Xestión de Usuarios e Tarefas - Aplicación creada con Vue 3 + Tailwind CSS

A.12. Resumo

- **Clases utilitarias de Tailwind CSS.** Permiten definir directamente no HTML propiedades como cores, tamaños, espazado, bordes, sombras, tipografía e distribución dos elementos, **reducindo a necesidade de CSS tradiciona**l.
- **Deseño responsive con Flexbox e Grid.** Tailwind facilita a **creación de interfaces que se adaptan a diferentes tamaños de pantalla** mediante clases responsive e sistemas de distribución como flex e grid.
- **Integración con Vue e clases dinámicas.** As clases de Tailwind poden combinarse **coa reactividade de Vue mediante :*class*,** permitindo modificar o aspecto dos elementos en función dos datos ou do estado da aplicación.
- **Compoñentes e reutilización.** A combinación de Vue e Tailwind permite crear compoñentes independentes e reutilizables, conseguindo **interfaces máis coherentes,** organizadas e fáciles de manter.